ETHEREUM PEERDAS · CODING SCHEME COMPARISON

1D-RS & 2D-RS Code Walkthrough

Implementation reference for 1D_RS_Galois.py and 2D_RS_Galois.py

---

Code walkthrough: `1D_RS_Galois.py` and `2D_RS_Galois.py`

This document explains what the two Reed-Solomon simulation scripts actually do — the mechanics, the maths behind each step, and worked numeric examples you can check by hand — plus the correctness checks that were run against them. It's a companion to 1D_vs_2D_RS_-comparison.pdf, which covers the conceptual comparison and how the results feed into the report.

Scope

Both files are **coding-scheme simulations**, not protocol implementations. The goal is to measure and compare the computational behaviour of the coding scheme itself (encode/decode cost, overhead, erasure-tolerance) under Ethereum's actual blob representation — not to reproduce PeerDAS's networking, subnet assignment, KZG proof generation, or gossip layer. Every place a real implementation detail was simplified away is called out explicitly in the module docstrings and repeated below, so the simplifications are a documented design decision rather than an oversight.

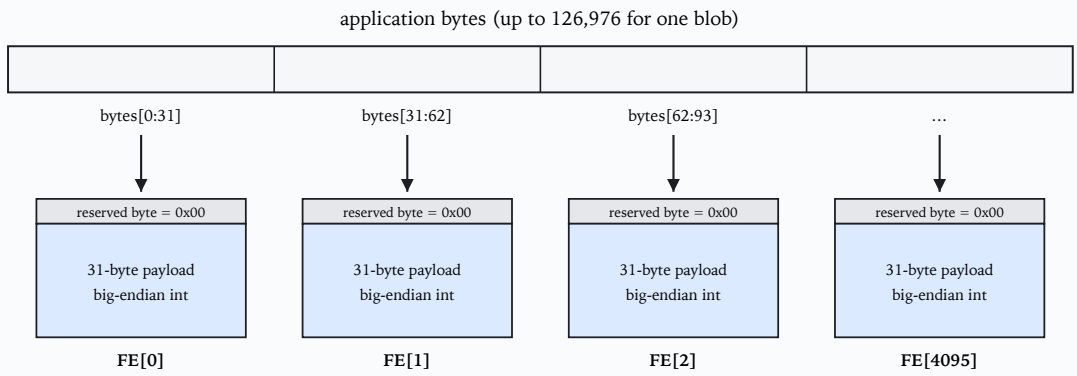
Shared foundation

`blob_generator.py`

Both scripts import this module rather than hand-rolling blob data. It encodes the real EIP-4844 constants and packing convention:

- `FIELD_ELEMENTS_PER_BLOB = 4096`
- `BLS_MODULUS` — the BLS12-381 scalar field order

- `bytes_to_blobs()` packs arbitrary application bytes at the real rate of 31 usable bytes per 32-byte field element (248 bits used out of 255, so every packed value is guaranteed canonical — i.e. less than the modulus)
- `blob_to_bytes()` is its exact inverse



int.from_bytes(word, "big") uses only 248 of 255 bits → always < BLS_MODULUS, i.e. canonical by construction — exactly bg.bytes_to_blobs(); the top byte of every field element is never used.

How EIP-4844 packs application bytes into blob field elements. Each 31-byte chunk becomes the payload of one 32-byte field element (bg.bytes_to_blobs()); the reserved top byte is why every field element is guaranteed less than the BLS12-381 modulus, with no range checks needed.

Both scripts start from `os.urandom()` application bytes, pack them via `bg.bytes_to_blobs()`, and — after the full encode/corrupt/decode/extract round trip — unpack the recovered blob back to bytes and compare against the original. This means "the demo passed" is checked at the *application data* level, not just at the level of raw field elements matching.

The field and the domain

Both files use the identical BLS12-381 scalar field:

```
P = 52435875175126190479447740508185965837690552500527637822603658699938581184513
GF = galois.GF(P)
```

`P - 1` is divisible by every power-of-two size the scripts use (64, 128, 4096, 8192), which is what makes NTTs of those sizes possible at all — both files assert this at import time. Concretely: an order-`n` NTT needs a primitive `n`-th root of unity `ω` in the field (an element with `ω^n = 1` and no smaller positive power equal to `1`); such an element exists exactly when `n` divides `P - 1`. Both scripts derive `ω` the same way:

```python
def root_of_unity(n, p):
    g = galois.primitive_root(p)   # generator of the whole group
    a = pow(g, (p - 1) // n, p)    # land on an n-th root
    assert pow(a, n, p) == 1 and pow(a, n // 2, p) != 1  # order is exactly n
    return a
```

WHAT THE NTT ACTUALLY COMPUTES, AND WHY IT'S FAST

A polynomial `f(x) = c₀ + c₁x + c₂x² + ... + c_{n-1}x^{n-1}` can be represented two equivalent ways: by its `n` coefficients, or by its values at any `n` distinct points. Evaluating `f` at `n` arbitrary points costs `O(n²)` field multiplications the naive way (n points × up to n terms each). The **Number Theoretic Transform** (the same idea as an FFT, but over a finite field instead of the complex numbers) is what `galois.ntt()` computes: it evaluates `f` at all `n` powers of a primitive `n`-th root of unity, `α⁰, α¹, ..., α^{n-1}`, simultaneously, in `O(n log n)` — by recursively splitting the polynomial into even- and odd-indexed coefficients and reusing partial sums, the way any radix-2 FFT does. `galois.intt()` is its exact inverse: given those `n` evaluations, recover the `n` coefficients, also in `O(n log n)`. This asymmetry — evaluate-everywhere-at-once in `O(n log n)` instead of `O(n²)` — is *the entire reason* RS-over-evaluation-form is practical at blob scale: at `n = 8192`, `O(n²)` is on the order of 67 million operations per extension; `O(n log n)` is on the order of 100,000.

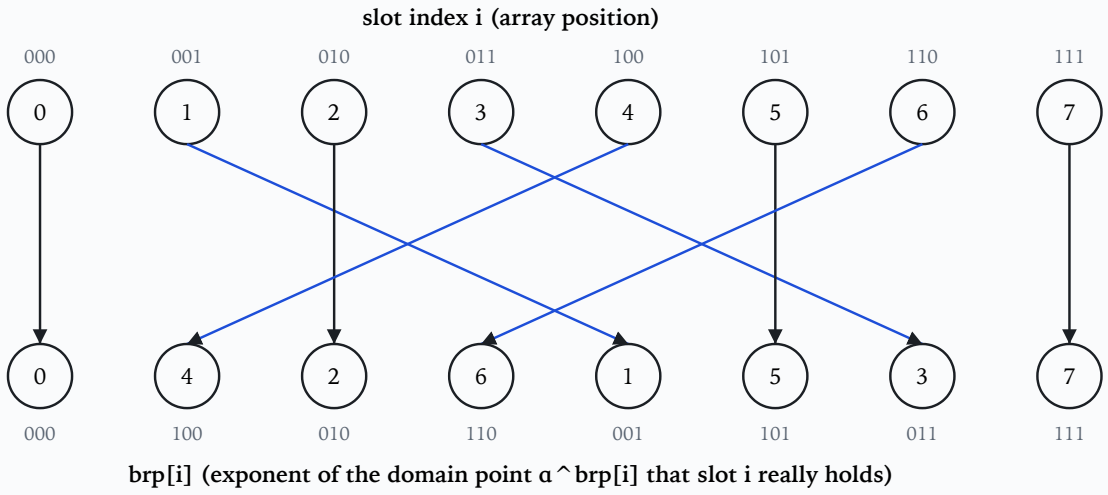
Both scripts were checked against this definition directly before anything else was built on top of it: `galois.ntt(coeffs, size=n, modulus=P)` was compared, coefficient by coefficient, against a brute-force Vandermonde evaluation `Σᵢ cᵢ · α^(i·j)` at `j = 0..n-1`, and the two matched exactly — confirming `galois.ntt` returns evaluations in **natural order** (index `j` holds `f(α^j)`), which is the assumption every other function in both files is built on.

Bit-reversal permuted domain ordering

Per `consensus-specs`, a blob's field elements are evaluations of a polynomial at the roots of unity, but not in the "natural" order `alpha^0, alpha^1, alpha^2, ...`. The domain itself is bit-reversal permuted: grid/codeword slot `i` corresponds to domain point `alpha^(bitreverse(i))`.

```python
def bit_reversal_permutation(n):
    bits = n.bit_length() - 1
    return [int(format(i, f"0{bits}b")[::-1], 2) for i in range(n)]

def bitreverse_permute(values, brp):
    return [values[brp[i]] for i in range(len(values))]
```



The bit-reversal permutation for an 8-point domain (bit_reversal_permutation(8)). Reading each index's binary right-to-left gives its true domain exponent — slot 1 (001) really holds the evaluation at α^4 (100), not α^1. Straight-down arrows (0, 2, 5, 7) are fixed points; crossing arrows (1↔4, 3↔6) are swapped pairs — a picture of why the permutation is its own inverse.

Bit-reversal is its own inverse (`brp[brp[i]] == i` — reversing an already- reversed bit string gets back the original), which is what lets `bitreverse_permute` do the conversion in *both* directions (grid-slot order → natural NTT order, and back) with one function: swap pairs like `1 ↔ 4` and `3 ↔ 6` in the diagram above undo each other, and fixed points like `0`, `2`, `5`, `7` (whose binary is a palindrome) don't move at all.

This ordering choice doesn't change *which* erasure patterns are recoverable (that's invariant under any consistent relabelling of positions) — it only changes which literal array index a given domain point lands in. It matters for standards conformance, not for the erasure-correcting math itself.

The core primitive: `interpolate_then_extend` / `restrict`

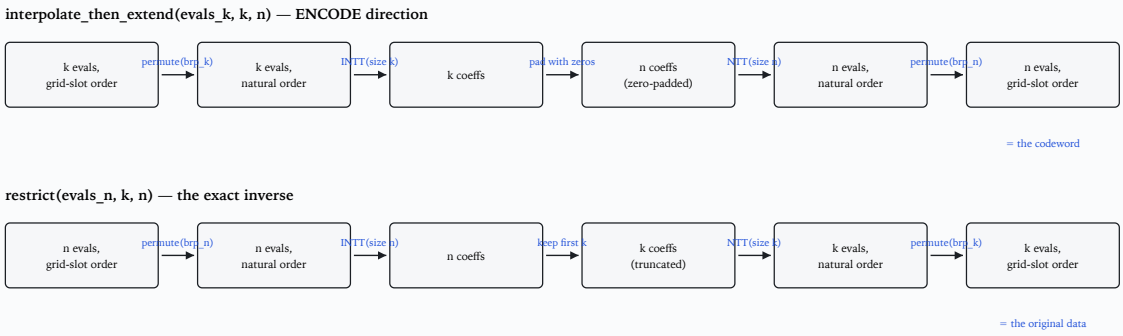
This is the single most important correctness fix relative to an earlier draft of the 2D file, and it's shared verbatim (modulo array-vs-matrix shape) between both scripts.

The standards point: per EIP-4844, a blob's field elements *are* the evaluations of a degree-`<4096` polynomial at the 4096th roots of unity — not the polynomial's coefficients. This is exactly why KZG can commit to a blob directly against a Lagrange-basis structured reference string, with no FFT needed at commitment time. Extending the blob (the RS "add redundancy" step) must therefore start by *interpolating* the known evaluations into coefficients, before evaluating on a larger domain — not by treating the raw data as coefficients and NTT-ing it directly (which was the bug in the earlier draft).

```python
def evals_from_coeffs(coeffs, n, brp_n, p):
    """Zero-pad ascending coefficients to n and evaluate via NTT
    (natural order), then reorder into grid-slot order."""
    padded = [int(c) for c in coeffs] + [0] * (n - len(coeffs))
    natural = [int(v) for v in galois.ntt(padded, size=n, modulus=p)]
    return bitreverse_permute(natural, brp_n)

def interpolate_then_extend(evals_cellorder_k, k, n, brp_k, brp_n, p):
    natural_k = bitreverse_permute([int(v) for v in evals_cellorder_k], brp_k)
    coeffs = [int(v) for v in galois.intt(natural_k, size=k, modulus=p)]
    return evals_from_coeffs(coeffs, n, brp_n, p)

def restrict(evals_cellorder_n, k, n, brp_k, brp_n, p):
    """Inverse of interpolate_then_extend."""
    natural_n = bitreverse_permute([int(v) for v in evals_cellorder_n], brp_n)
    coeffs_n = [int(v) for v in galois.intt(natural_n, size=n, modulus=p)]
    coeffs_k = coeffs_n[:k]
    natural_k = [int(v) for v in galois.ntt(coeffs_k, size=k, modulus=p)]
    return bitreverse_permute(natural_k, brp_k)
```



The two core operations both scripts share: interpolate_then_extend() (encode direction, top) and its exact inverse restrict() (extraction direction, bottom). Every arrow is one line of code; the worked numeric example above runs this exact sequence over a toy field so each step can be checked by hand.

`interpolate_then_extend` is the real FFT-based RS extension referenced by the community writeup on Reed-Solomon erasure code recovery via FFTs (`ethresear.ch`): interpolate → zero-pad → evaluate on the bigger domain, all in `O(n log n)` . `restrict` is its exact inverse, used during extraction once a codeword is fully known and needs to be brought back down to the original `k` evaluations — it's *not* simply "inverse-NTT and truncate", because the recovered values are evaluations, not coefficients; recovering the true `k` original evaluations requires one more forward NTT at size `k` after truncating to the known-degree coefficients.

A FULLY WORKED EXAMPLE (TINY FIELD, SO YOU CAN CHECK EVERY STEP BY HAND)

To make this concrete rather than purely symbolic, here is the exact pipeline run over a small field, `P = 17`, with `k = 2` and `n = 4` (instead of `k = 4096` / `n = 8192`) — small enough to verify with a calculator, using the same code paths as the real scripts.

`17 − 1 = 16` is divisible by 4, so a primitive 4th root of unity exists: `g = 3` is a primitive root of `17`, and `α = g^((17−1)/4) mod 17 = 3⁴ mod 17 = 13`. Check: `13⁴ mod 17 = 1` ✓.

Take a tiny "blob" of `k = 2` evaluations, in grid-slot order: `data = [5, 9]`. For `n = k = 2`, the bit-reversal permutation is the identity, so grid-slot order already *is* natural order here: these are `f(α₂⁰) = 5` and `f(α₂¹) = 9` for some degree-`<2` polynomial `f`.

Encode (`interpolate_then_extend(data, k=2, n=4)`):

1. `INTT(size=2)` on `[5, 9]` → coefficients `[7, 15]`, i.e. `f(x) = 7 + 15x`. (Check: `f(α₂⁰) = f(1) = 7+15 = 22 mod 17 = 5` ✓, and `f(α₂¹) = f(16) = 7 + 15·16 mod 17 = 7+240 mod 17 = 9` ✓.)
2. Zero-pad to length 4: `[7, 15, 0, 0]`.
3. `NTT(size=4)` on the padded coefficients → natural-order evaluations `[5, 15, 9, 16]`, i.e. `f(α⁰)=5, f(α¹)=15, f(α²)=9, f(α³)=16`.
4. Permute into grid-slot order via `brp₄ = [0,2,1,3]` (swap slots 1 and 2, fixed points 0 and 3) → **the codeword: `[5, 9, 15, 16]`**.

You can sanity-check step 4 directly: slot `1` should hold `f(α^(brp₄[1])) = f(α²) = 9` — and it does.

Decode (recovering from one erasure): erase slot `2` (value `15`), so the received line is `[5, 9, 0, 16]`.

1. Slot 2's domain exponent is `brp₄[2] = 1`, so the erased point is `α¹ = 13`. The erasure-locator polynomial is `Z(x) = x − 13 ≡ x + 4`.
2. Zero-pad `Z`'s coefficients `[4, 1]` to length 4 and NTT → `Z_evals = [5, 0, 3, 8]` (note `Z_evals[1] = 0` exactly, confirming `Z` really does vanish at the erased point's natural-order index).
3. Convert the received line to natural order: `[5, 0, 9, 16]` (erased slot 2 → natural index `brp₄[2]=1`, already zero).
4. Pointwise multiply by `Z_evals`: `[5·5, 0·0, 9·3, 16·8] mod 17 = [8, 0, 10, 9]`.
5. `INTT(size=4)` on that product → coefficients `[11, 16, 15, 0]`, i.e. the product polynomial is `11 + 16x + 15x²` (its degree is `< 3`, as it must be: degree-`<2` original × degree-1 `Z`).
6. Divide by `Z(x) = x + 4`: quotient `= 7 + 15x`, remainder `= 0`. The zero remainder is the proof the erasure count was within budget — a nonzero remainder is exactly what the code's `WARNING` check is watching for.

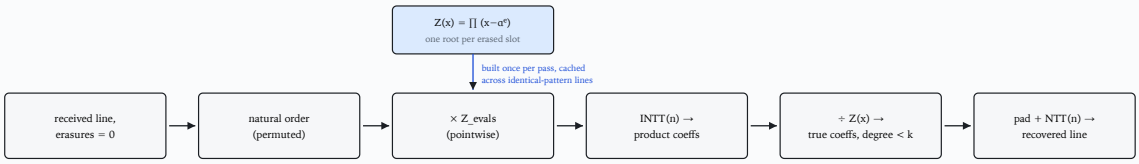
7. The recovered coefficients `[7, 15]` are **exactly** what step 1 of encoding produced — the original polynomial, recovered from 3 out of 4 values. Re-evaluating them (pad to 4, NTT, permute) reproduces the full codeword `[5, 9, 15, 16]`, erasure included.

This is the literal sequence `recover_line_ntt()` / `recover_codeword_ntt()` runs in both scripts, just at `k=4096, n=8192` (1D-RS) or `k=64, n=128` per line (2D-RS) instead of `k=2, n=4`.

`1D_RS_Galois.py` — the PeerDAS baseline

This is the degenerate, single-axis case: the blob is treated as **one** codeword, never reshaped into a grid.

FUNCTION	ROLE
`encode_blob(data_vector)`	`interpolate_then_extend` the 4096 evaluations to 8192 (`k=4096, n=8192`) — a single call, one NTT pair.
`build_erasure_locator(erased_cells, n, k, alpha, brp_n, p)`	Builds the monic erasure-locator polynomial `Z(x)`, with a root at the domain point of every erased slot, plus its evaluations. Returns `(None, None)` if `len(erased_cells) > n - k` — more erasures than the code's redundancy budget can ever resolve.
`recover_codeword_ntt(...)`	The FFT-based erasure-decode step (worked example above): pointwise-multiply the received (zeroed-at-erasures) codeword by `Z`'s evaluations, inverse-NTT, divide out `Z(x)` via ordinary polynomial long division to recover the true degree-`<k` coefficients, then re-evaluate (NTT) on the full `n`-domain to fill in every erased slot at once.
`decode(corrupted_vector, mask, ...)`	The one-shot decision point: if the erasure count is within budget, decode and return `(recovered, True)`; if not, return `(corrupted_vector, False)` with an explicit failure message. **There is no fallback path** — 1D-RS has no second axis to lean on.
`restrict(...)`	Used at extraction to bring the recovered 8192-length codeword back down to the original 4096 evaluations.

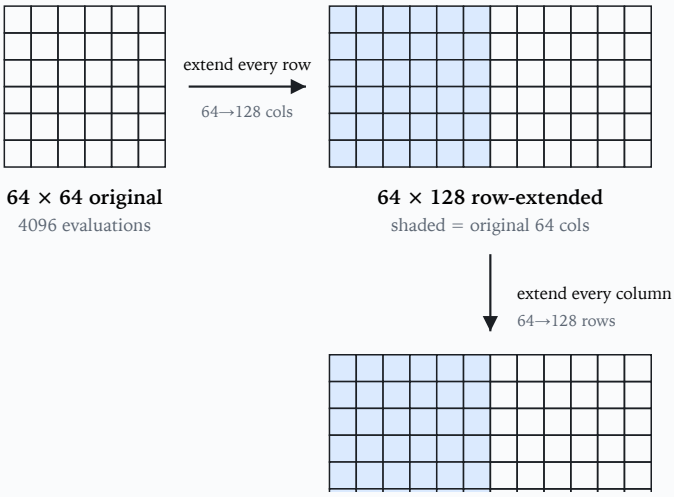


Recovering one erased line — 1D-RS's whole codeword, or one row/column of 2D-RS. The erasure-locator polynomial Z(x) is built once from the erased positions and reused across every identically-patterned line in a pass — this caching is what keeps decode_pass() fast in the common (whole-row/whole-column) case.

The demo (`__main__` block) runs: pack real bytes → blob → encode → scatter-erase 200 samples → decode → restrict → unpack back to bytes → compare. It then runs three random-adversarial-rate trials (5%, 15%, 55%) to show recoverable-vs-unrecoverable behaviour and its associated timing.

`2D_RS_Galois.py` — the product code

The blob is reshaped into a 64×64 grid and extended along both axes independently (64→128 rows, 64→128 columns), a 4× total overhead. **This is an explicit, documented design choice** — matching PeerDAS's actual 2× *total* overhead exactly would need non-power-of-two per-axis sizes, which complicates using a clean NTT. It's reported as its own datapoint against 1D-RS's 2×, not treated as a bug.



Encoding the 2D grid: every row is extended first (64→128 columns), then every column of that result is extended (64→128 rows), producing the full 128×128 codeword. The shaded region tracks where the original 64×64 data sits after each stage.

Why every row and every column is independently a valid codeword

`encode_blob` extends every row first (`k=64 cols → n=128 cols`), then every column of the row-extended grid (`k=64 rows → n=128 rows`). This is more than a bookkeeping convenience — it's what makes the grid a genuine RS **product** code. Concretely: the 64×64 array of original values defines the coefficients `c_{i,j}` (`0 ≤ i,j < 64`) of a bivariate polynomial

```
P(x, y) = Σᵢ Σⱼ c_{i,j} · x^i · y^j
```

with degree `< 64` in `x` and, independently, degree `< 64` in `y` . Fix any value of `y` — call it `y₀` , whether or not `y₀` was one of the original 64 domain points:

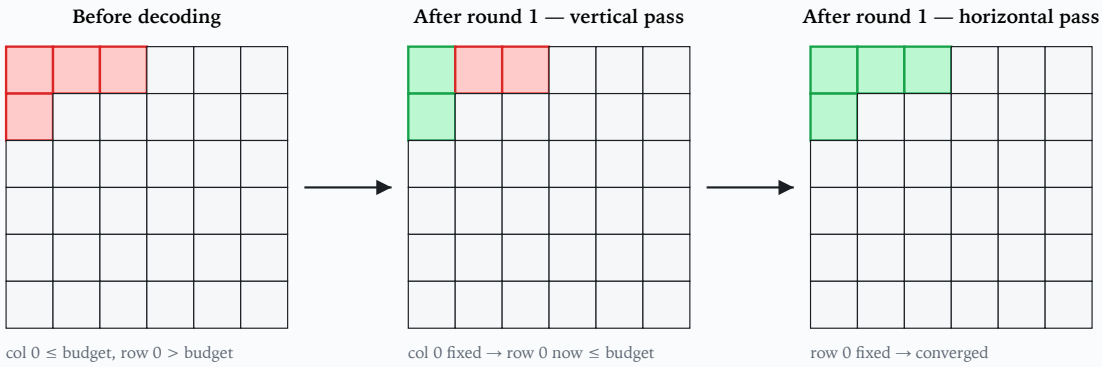
```
P(x, y₀) = Σᵢ ( Σⱼ c_{i,j} · y₀^j ) · x^i
```

is *still* a polynomial in `x` alone, and its degree is still `< 64` — the inner sum over `j` just collapses each row's coefficients down to a single number per `i` ; it can't raise the degree in `x` . The same argument applies with the roles of `x` and `y` swapped. That's the whole proof: **every** row of the fully-extended 128×128 grid — not just the original 64 — is a valid degree-`<64` codeword in the column direction, and **every** column is a valid degree-`<64` codeword in the row direction, regardless of whether that row/column itself came from an original domain point or an "extension" point. This is why `decode_pass` treats every line identically with no special case for "was this row/column part of the original data" — and it was confirmed empirically (exact recovery in every test run, including scattered erasure patterns that touch both original and parity rows/ columns).

The decoder: mask-based and iterative, not fixed-two-pass

FUNCTION	ROLE
`build_erasure_locator`, `recover_line_ntt`	Same FFT-based erasure-decode machinery as the 1D file (worked example above), applied to one row or one column at a time.
`decode_pass(grid, mask, axis, ...)`	Runs one pass along one axis. For **every** line in that axis, it reads that line's *own* currently-erased cell positions straight from the `mask` (not a single pattern assumed shared across all lines), builds (or reuses, via a `locator_cache` keyed by the erased-position tuple) that line's locator polynomial, and recovers it if the erasure count is within that axis's `n − k` budget. Lines that share an identical pattern — the common case of a handful of whole rows/columns wiped out — still share one cached locator, so that case isn't slowed down by the per-line generalisation.
`decode_iterative(...)`	Alternates vertical and horizontal passes, **repeatedly**, until either every erasure is cleared or a full round makes no further progress (capped at `max_rounds=20` as a safety net, never actually needed in testing — 1–2 rounds always sufficed). This loop is the actual mechanism that gives a product code its extra erasure-correcting strength: a cell that neither axis alone could recover in round 1 (because its row *and* its column each individually exceeded their own budget) can become recoverable in round 2 once the other axis has filled in enough of its neighbours.
`random_erasure_mask(n_rows, n_cols, rate, rng)`	Generates a per-cell i.i.d. adversarial sampling pattern, for the report's "varying adversarial sampling rates" methodology — as opposed to only ever modelling a few whole rows/columns being withheld, which is all a *fixed* two-pass decoder could ever represent.

Here is exactly what that iteration buys you, worked through on a small illustrative grid (`N=6` per line, budget `n−k=2` — not the real 64/128 numbers, just small enough to draw and follow):



illustrative N=6, budget=n-k=2 per line — same mechanism as decode_pass()/decode_iterative(), not literal output

A cell that neither its row nor its column could recover alone becomes recoverable once the other axis runs first. Column 0 clears within its own budget on the vertical pass; that drops row 0's erasure count enough for the horizontal pass, in the same round, to clear it too. Illustrative N=6, budget=2 — the same shape of example as test 3 in the correctness suite below, not literal script output.

Column 0 has 2 erasures — within its own budget of 2, so it recovers on the very first vertical pass regardless of what row 0 is doing. Row 0, read in isolation *before* that happens, has 3 erasures (cells `(0,0)`, `(0,1)`, `(0,2)`) — over its budget of 2, so `build_erasure_locator` returns `(None, None)` for it and `decode_pass` leaves it marked as still erased. But `decode_iterative` doesn't stop there: because the vertical pass runs *first* within the same round, column 0's fix lands in the grid before the horizontal pass reads row 0's erasure count — and by then, cell `(0,0)` is no longer erased, so row 0's count has dropped to exactly 2, within budget. The horizontal pass, in the very same round, clears it. Neither axis's own `n-k` budget "grew" — the pattern simply stopped looking like 3 erasures to row 0 once column 0 did its job first. This is precisely the mechanism exercised by test 3 of the small-field test suite described below, and it's the concrete answer to "where does 2D-RS's extra fault tolerance actually come from" (elaborated further, with the 1D-RS contrast, in the comparison document).

The demo runs the same pack → encode → corrupt → decode → extract → unpack-and-compare round trip as the 1D file, using a small deterministic corruption (two whole rows + one whole column) for the walkthrough, then two random-adversarial-rate trials (15%, 35%) for comparison against the 1D file's own trials.

Correctness verification performed

Small-field sanity suite (`P = 65537`, `K = 4`, `N = 8`, run standalone before trusting the real-field version):

1. No-erasure round trip: exact match.

2. Whole-row/whole-column erasure (the old fixed-pattern case): recovered exactly in 1 round.
3. Scattered erasure engineered so a column exceeds its axis's budget until the *other* axis fixes it first (the same shape of example as the diagram above): recovered exactly, demonstrating the iterative mechanism actually engages rather than just being unreachable dead code.
4. 15 independent random-rate (20%) trials: 15/15 fully recovered.
5. A fully-erased grid (deliberately impossible): decoder correctly reports remaining erasures rather than silently returning corrupted data.

Full blob-scale runs (real BLS12-381 field, real 4096-element blobs packed from `os.urandom()` application bytes, via `blob_generator.py`):

- `1D_RS_Galois.py`: encoded grid, blob data, and unpacked application bytes all matched exactly after a 200-sample scattered-erasure decode; 5% and 15% random-rate trials recovered fully; the 55% trial (above the 50% budget) failed immediately and explicitly, as expected — no silent corruption.
- `2D_RS_Galois.py`: encoded grid, blob data, and unpacked application bytes all matched exactly after the two-whole-row/one-whole-column corruption (converged in 1 round); 15% and 35% random-rate trials both recovered fully.

No correctness issues were found in either file during this review.

Documented simplifications (from the module docstrings)

Both files:

- Model an **erasure channel only** (missing samples), not arbitrary symbol errors — this matches how PeerDAS samples are actually used (a peer either returns a sample or it doesn't) and matches what NTT-based erasure decoding corrects.
- Recover a line's/codeword's coefficients via **ordinary polynomial long division** (`Prod_poly // Z_poly`), not a Newton-iteration power-series inversion. Encode is a clean `O(n log n)` either way; decode is `O(n log n)` for the NTT/INTT calls plus that division's cost — a genuine FFT-based erasure decoder, but not the fully NTT-based `O(n log² n)` decoder real systems can use.

`2D_RS_Galois.py` additionally:

- Doubles **both** axes independently for 4× total overhead, rather than matching PeerDAS's 2× total (see above).

`1D_RS_Galois.py` additionally documents an empirically observed cost asymmetry worth carrying into the comparison document: dividing **one** large (~8192-degree) polynomial scales noticeably worse with erasure count than dividing **many small** (~128-degree) polynomials for a similar total number of erasures, because the `galois` library's polynomial division is school-

book-style and its cost grows with the divisor's degree. Measured on this machine: ~10s at 200 erasures, ~70s at 2000, ~78s at 4096 for the 1D file's single big division, versus low-single-digit seconds total for the 2D file's many small per-line divisions at a comparable total erasure count. This is flagged as a real datapoint for the report's computational- complexity comparison, not an implementation flaw — see the measured chart in the comparison document.

Running the scripts

```
pip install galois numpy
python3 1D_RS_Galois.py
python3 2D_RS_Galois.py
```

Both are self-contained demos (`if __name__ == "__main__":`) that print a full step-by-step walkthrough with timings; neither takes command-line arguments. Expect the full-field runs to take on the order of a few minutes each, dominated by the polynomial-division cost discussed above — this is a correctness/completeness trade-off made deliberately for this stage of the project (a from-scratch, standards-faithful simulation you can read top-to-bottom), not a claim about production-grade performance.
